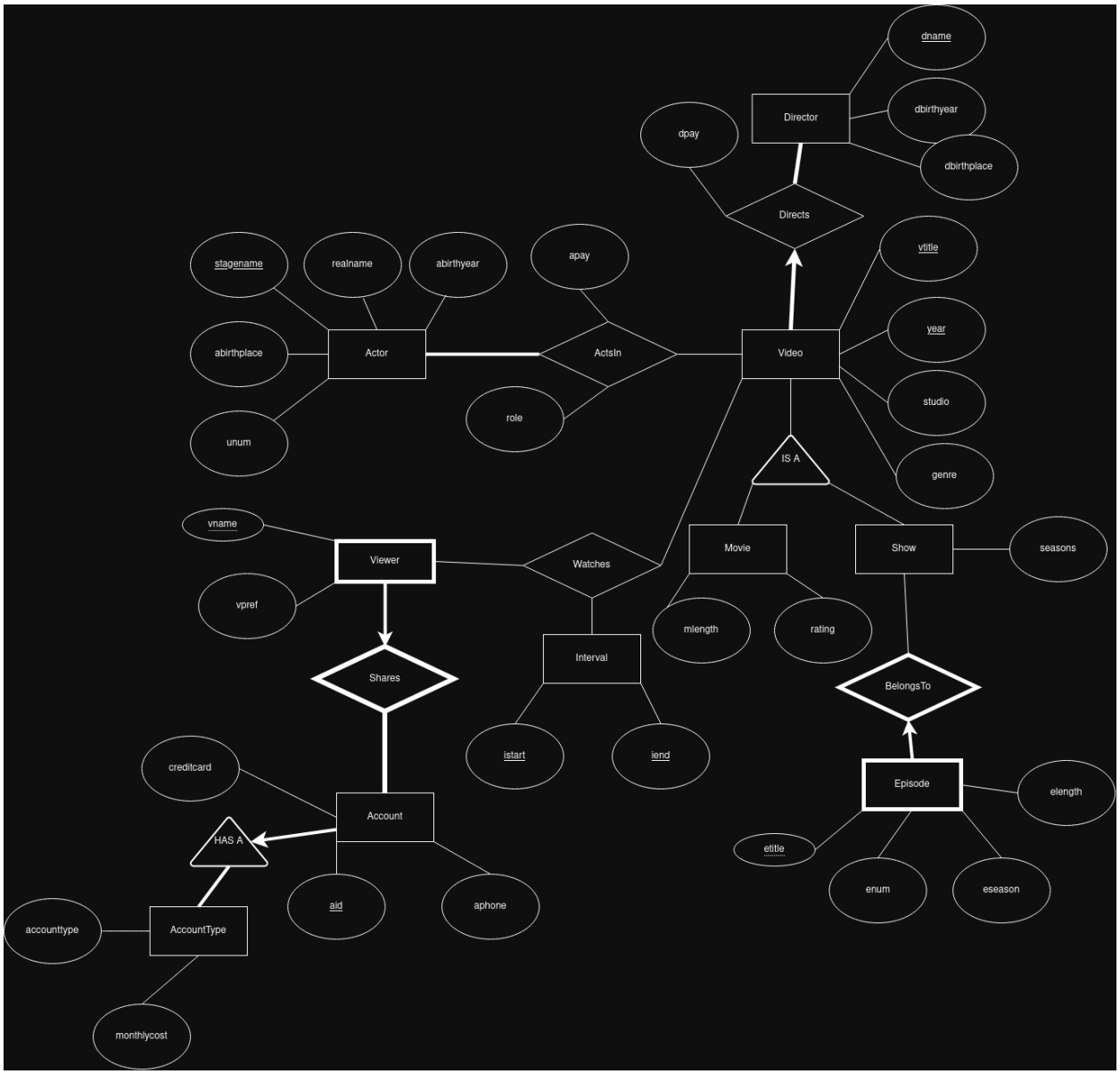
Lab #3

Group #4
Logan Yu - py22kg@brocku.ca
Gregory Sampson - gs22tk@brocku.ca
Adam Chorzepa - oe23lo@brocku.ca

Question 1:



The total participation constraints here are that every Account must have exactly one AccountType, since otherwise we will not know how much to bill that account. Viewers must be part of exactly one Account, but an account can have zero or multiple viewers.

Actors must act in at least one Video. A video does not have to be a movie or a Show (since documentaries and other miscellaneous videos can appear). A video must be directed by exactly one director. Episodes must belong to exactly one show, and a show must have at least one episode.

Question 2:

Actors(stagename, realname, abirthyear, abirthplace, unum)

Directors(dname, dbirthyear, dbirthplace)

VideoD(vtitle, year, studio, genre, dname, dpay)

Movie(vtitle, year, mlength, rating)

Show(vtitle, year, seasons)

ActsIn(stagename, vtitle, year, apay, role)

EpisodeOf(etitle, vtitle, year, enum, eseason, elength)

AccountType(account_type, monthly_cost)

Account(aid, aphone, account_type, credit_card)

ViewerShares(vname, aid, vpref)

Watches(vname, aid, vtitle, year, istart, iend)

Question 3:

1. Actor(stagename, realname, abirthyear, abirthplace, unum)
   - Functional Dependencies:
     - stagename → realname, abirthyear, abirthplace, unum
     - unum → stagename, realname, abirthyear, abirthplace (when not null)

   - Normal Form: BCNF
     - For every non-trivial functional dependency that holds, the determinant is a superkey.
     - Attribute stagename is a candidate key and determines all other attributes
     - Unum is unique when present, therefore it is also a candidate key. Any dependency with Unum on the left hand side satisfies the BCNF condition.

2. Director(dname, dbirthyear, dbirthplace)
   - Functional Dependencies:
     - dname → dbirthyear, dbirthplace

   Normal Form: BCNF
     - For every non-trivial functional dependency that holds, the determinant is a superkey.

- dname is the candidate key for the relation
- Every non-trivial dependency has a key on the left hand side, therefore it satisfies the BCNF condition.

3. VideoD(vtitle, year, studio, genre, dname, dpay)
   - Functional Dependencies:
     - (vtitle, year) → studio, genre, dname, dpay

   - Normal Form: BCNF
     - For every non-trivial functional dependency that holds, the determinant is a superkey.
     - (vtitle, year) → studio, genre, dname, dpay is the functional dependency, so (vtitle, year) is the candidate key
     - Every non-trivial dependency has a key on the left hand side, therefore it satisfies the BCNF condition.

4. Movie(vtitle, year, mlength, rating)
   - Functional Dependencies:
     - (vtitle, year) → mlength, rating

   - Normal Form: BCNF
     - For every non-trivial functional dependency that holds, the determinant is a superkey.
     - (vtitle, year) → mlength, rating is the only functional dependency, so (vtitle, year) is the candidate key.
     - Every non-trivial dependency has a key on the left hand side, therefore it satisfies the BCNF condition.

5. Show(vtitle, year, seasons)
   - Functional Dependencies:
     - (vtitle, year) → seasons

   - Normal Form: BCNF
     - For every non-trivial functional dependency that holds, the determinant is a superkey.
     - (vtitle, year) is the candidate key.
     - Every non-trivial dependency has a key on the left hand side, therefore it satisfies the BCNF condition.

6. ActsIn(stagename, vtitle, year, apay, role)
   - Functional Dependencies:

        - (stagename, vtitle, year) → apay, role

    - Normal Form: BCNF
        - For every non-trivial functional dependency that holds, the determinant is a superkey.
        - (stagename, vtitle, year) is the candidate key.
        - Every non-trivial dependency has a key on the left hand side, therefore it satisfies the BCNF condition.

7. EpisodeOf(etitle, enum, eseason, elength, vtitle, year)
    - Functional Dependencies:
        - (etitle, vtitle, year) → enum, eseason, elength
            - No two episodes share the same title from the same show

        - (enum, eseason, vtitle, year) → etitle, elength
            - No two episodes have the same episode number and eseason in a show

    - Normal Form: BCNF
        - For every non-trivial functional dependency that holds, the determinant is a superkey.
        - (etitle, vtitle, year) and (enum, eseason, vtitle, year) form candidate keys
        - Every non-trivial dependency has a key on the left hand side, therefore it satisfies the BCNF condition.

8. AccountType(account_type, monthly_cost)
    - Functional Dependencies:
        - account_type --> monthly_cost
            - accounts with the same type have the same monthly cost

    - Normal Form: BCNF
        - For every non-trivial functional dependency that holds, the determinant is a superkey.
        - account_type is the candidate key
        - Every non-trivial dependency has a key on the left hand side, therefore it satisfies the BCNF condition.

9. Account(aid, aphone, account_type, credit_card)
    - Functional Dependencies:
        - aid → aphone, account_type, credit_card
        - aphone → aid
        - credit_card → aid

    - Normal Form: BCNF

- For every non-trivial functional dependency that holds, the determinant is a superkey.
- aid, aphone, and credit_card are candidate keys.
- Every non-trivial dependency has a key on the left hand side, therefore it satisfies the BCNF condition.

*Account(aid, aphone, account_type, monthly_cost, credit_card) is not in BCNF because account_type → monthly_cost holds, but account_type is not a superkey.

Due to this, it has been decomposed into AccountType and Account:
- AccountType(account_type, monthly_cost)
- Account(aid, aphone, account_type, credit_card)

The decomposition is lossless-join because account_type is a key in AccountType and appears in both relations.
It is dependency-preserving because the FD account_type → monthly_cost is valid in AccountType

Question 4:

```
-- ------------------------------------------------------------
-- Actor
-- Note: unum is unique but nullable (some actors have no union number).
-- ------------------------------------------------------------
CREATE TABLE Actor (
   stagename   CHAR(30),
   realname    CHAR(30),
   abirthyear  INTEGER,
   abirthplace CHAR(30),
   unum        INTEGER,
   PRIMARY KEY (stagename),
   UNIQUE (unum)
)

-- ------------------------------------------------------------
-- Director
-- ------------------------------------------------------------
CREATE TABLE Director (
   dname       CHAR(30),
   dbirthyear  INTEGER,
   dbirthplace CHAR(30),
   PRIMARY KEY (dname)
)
```

```
-- ---------------------------------------------------------
-- VideoD  (combines Video and Directs)
-- dname is NOT NULL because every video must have exactly one director.
-- year CHECK constraint: must be between 1900 and 2026.
-- ---------------------------------------------------------
CREATE TABLE VideoD (
    vtitle  CHAR(30),
    year    INTEGER CHECK (year BETWEEN 1900 AND 2026),
    studio  CHAR(30),
    genre   CHAR(30),
    dname   CHAR(30) NOT NULL,
    dpay    INTEGER,
    PRIMARY KEY (vtitle, year),
    FOREIGN KEY (dname) REFERENCES Director
        ON DELETE NO ACTION
        ON UPDATE CASCADE
)

-- ---------------------------------------------------------
-- Movie  (ISA subtype of VideoD)
-- rating CHECK constraint: must be an integer between 0 and 100.
-- ---------------------------------------------------------
CREATE TABLE Movie (
    vtitle  CHAR(30),
    year    INTEGER,
    mlength INTEGER,
    rating  INTEGER CHECK (rating BETWEEN 0 AND 100),
    PRIMARY KEY (vtitle, year),
    FOREIGN KEY (vtitle, year) REFERENCES VideoD
        ON DELETE CASCADE
        ON UPDATE CASCADE
)

-- ---------------------------------------------------------
-- Show  (ISA subtype of VideoD)
-- ---------------------------------------------------------
CREATE TABLE Show (
    vtitle  CHAR(30),
    year    INTEGER,
    seasons INTEGER,
    PRIMARY KEY (vtitle, year),
    FOREIGN KEY (vtitle, year) REFERENCES VideoD
        ON DELETE CASCADE
        ON UPDATE CASCADE
```

```
)

-- ------------------------------------------------------------
-- ActsIn
-- ------------------------------------------------------------
CREATE TABLE ActsIn (
   stagename   CHAR(30),
   vtitle      CHAR(30),
   year        INTEGER,
   apay        INTEGER,
   role        CHAR(30),
   PRIMARY KEY (stagename, vtitle, year),
   FOREIGN KEY (stagename) REFERENCES Actor
      ON DELETE CASCADE
      ON UPDATE CASCADE,
   FOREIGN KEY (vtitle, year) REFERENCES VideoD
      ON DELETE CASCADE
      ON UPDATE CASCADE
)

-- ------------------------------------------------------------
-- EpisodeOf  (combines Episode and BelongsTo)
-- vtitle and year are NOT NULL (total participation of Episode in BelongsTo).
-- (enum, eseason, vtitle, year) is UNIQUE because no two episodes of the
-- same show may share the same (episode number, season number) combination.
-- Note: the constraint that eseason must be between 1 and the show's seasons
-- count requires a trigger and is deferred to Part 2.
-- ------------------------------------------------------------
CREATE TABLE EpisodeOf (
   etitle  CHAR(30),
   enum    INTEGER,
   eseason INTEGER,
   elength INTEGER,
   vtitle  CHAR(30) NOT NULL,
   year    INTEGER  NOT NULL,
   PRIMARY KEY (etitle, vtitle, year),
   UNIQUE (enum, eseason, vtitle, year),
   FOREIGN KEY (vtitle, year) REFERENCES Show
      ON DELETE CASCADE
      ON UPDATE CASCADE
)

-- ------------------------------------------------------------
-- AccountType  (new table extracted from Account for BCNF)
```

```
-- Stores the two possible account types and their monthly costs.
-- Because all accounts of the same type share the same monthly cost,
-- the FD  account_type → monthly_cost  must be captured here.
-- ------------------------------------------------------------
CREATE TABLE AccountType (
    account_type    CHAR(10),
    monthly_cost    DECIMAL(10,2),
    PRIMARY KEY (account_type),
    CHECK (account_type IN ('Standard', 'Premium'))
)

-- ------------------------------------------------------------
-- Account
-- aphone is UNIQUE because a phone number can belong to only one account.
-- credit_card is UNIQUE because a credit card can belong to only one account.
-- account_type is NOT NULL (every account has an account type).
-- Note: The constraint that Standard accounts may have at most one viewer
-- requires a trigger and is deferred to Part 2.
-- ------------------------------------------------------------
CREATE TABLE Account (
    aid             INTEGER,
    aphone          CHAR(10),
    account_type    CHAR(10) NOT NULL,
    credit_card     CHAR(16),
    PRIMARY KEY (aid),
    UNIQUE (aphone),
    UNIQUE (credit_card),
    FOREIGN KEY (account_type) REFERENCES AccountType
        ON DELETE NO ACTION
        ON UPDATE CASCADE
)

-- ------------------------------------------------------------
-- ViewerShares  (combines Viewer and Shares)
-- aid is NOT NULL (total participation of Viewer in Shares).
-- ------------------------------------------------------------
CREATE TABLE ViewerShares (
    vname   CHAR(30),
    aid     INTEGER NOT NULL,
    vpref   CHAR(30),
    PRIMARY KEY (vname, aid),
    FOREIGN KEY (aid) REFERENCES Account
        ON DELETE CASCADE
        ON UPDATE CASCADE
```

```
)

-- ------------------------------------------------------------
-- Watches
-- iend >= istart: enforced with CHECK constraint.
-- Note: The constraint that a viewer cannot watch more than one video
-- at the same time (non-overlapping intervals per viewer) requires a
-- trigger and is deferred to Part 2.
-- ------------------------------------------------------------
CREATE TABLE Watches (
    vname   CHAR(30),
    aid     INTEGER,
    vtitle  CHAR(30),
    year    INTEGER,
    istart  TIMESTAMP,
    iend    TIMESTAMP,
    PRIMARY KEY (vname, aid, vtitle, year, istart, iend),
    CHECK (iend >= istart),
    FOREIGN KEY (vname, aid) REFERENCES ViewerShares
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (vtitle, year) REFERENCES VideoD
        ON DELETE CASCADE
        ON UPDATE CASCADE
)
```